

AUTOMATION TESTING PRACTICE WEBSITE – CAPSTONE PROJECT REPORT

1. Project Title

Automation Testing of Automation Practice Website using Selenium WebDriver, Java, TestNG, Maven, Jenkins, GitHub and Extent Reports

2. Project Overview

The Automation Testing Practice Website Automation Framework is developed to automate the testing of various web elements and functionalities available in the Automation Practice Website.

The framework is built using Selenium WebDriver with Java and follows the Page Object Model (POM) design pattern. Test execution is managed through TestNG, dependencies are maintained using Maven, source code is managed using GitHub, and continuous integration is achieved through Jenkins. Extent Reports are generated for detailed execution reporting.

The project validates different functionalities such as GUI Elements, Date Pickers, File Upload, Pagination Tables, Forms, Footer Links, Tabs, Dynamic Buttons, Alerts, Mouse Actions, Labels, Links, and Shadow DOM Components.

3. Project Objective

The primary objectives of this project are:

- Automate repetitive manual testing activities.
- Improve test execution speed.
- Increase test coverage.
- Reduce human errors.
- Generate detailed execution reports.
- Detect defects early.
- Integrate automation with Jenkins CI/CD.
- Improve application quality and reliability.

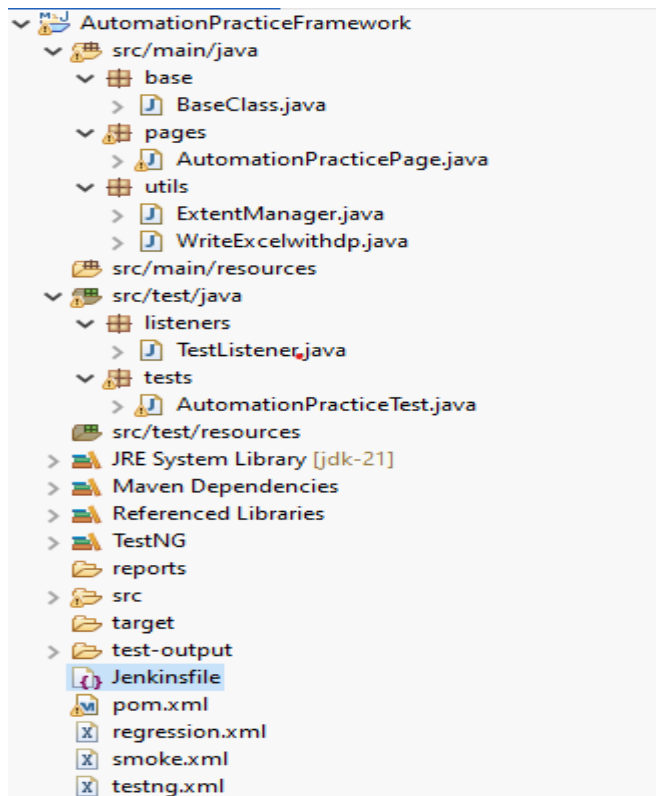
4. Technologies Used

Technology	Purpose
Java	Programming Language
Selenium WebDriver	Browser Automation
TestNG	Test Execution Framework
Maven	Dependency Management
Jenkins	Continuous Integration
GitHub	Version Control
Extent Reports	Test Reporting
Page Object Model (POM)	Framework Design Pattern

5. Framework Architecture

The project follows the **Page Object Model (POM)** design pattern where page objects, test scripts, utilities, listeners, and reporting components are separated into different packages for better maintainability and reusability.

Project Structure



Package Description

Base Package

BaseClass.java

This class serves as the foundation of the framework.

Responsibilities

- Browser Initialization
- WebDriver Setup
- Launch Application URL
- Browser Configuration
- Setup and Teardown Methods

Advantages

- Reusable Browser Setup
 - Reduced Code Duplication
 - Centralized Configuration
-

Pages Package

AutomationPracticePage.java

This page class contains all web element locators and actions related to the Automation Testing Practice Website.

Functionalities Automated

Unit 1 – GUI Elements

- Name Field
- Radio Buttons
- Checkboxes
- Sorted List

Unit 2 – Date Pickers

- Date Picker 1
- Date Picker 2
- Date Picker 3

Unit 3 – Upload Files

- File Upload Functionality

Unit 4 – Pagination Web Table

- Table Validation
- Pagination Handling

Unit 5 – Forms

- Form Submission
- Validation

Unit 6 – Footer Links

- Footer Navigation

Unit 7 – Tabs

- Wikipedia Tab
- New Tab Handling

Dynamic Buttons

- Dynamic Button Click Actions

Unit 8 – Alerts and Popups

- Alert Handling
- Confirmation Alert
- Prompt Alert

Unit 9 – Mouse Hover

- Mouse Hover
- Double Click
- Drag and Drop
- Slider
- Scrolling Dropdown

Unit 10 – Labels and Links

- Link Verification
- Label Validation

Unit 11 – Shadow DOM

- Shadow DOM Element Interaction

Utils Package

ExtentManager.java

Responsible for generating Extent Reports.

Features

- Create HTML Report
 - Log Test Results
 - Capture Execution Status
 - Generate Professional Reports
-

WriteExcelwithdp.java

Responsible for Data Driven Testing.

Features

- Read Test Data
 - Write Test Results
 - Excel Integration
 - Data Provider Support
-

Listeners Package

TestListener.java

Implements TestNG Listener functionality.

Responsibilities

- Track Test Execution
 - Capture Pass Status
 - Capture Fail Status
 - Capture Skip Status
 - Generate Logs
 - Trigger Screenshots on Failure
-

Tests Package

AutomationPracticeTest.java

Contains all automated test cases for the Automation Testing Practice Website.

Test Modules Covered

Unit	Module
Unit 1	GUI Elements
Unit 2	Date Pickers
Unit 3	Upload Files
Unit 4	Pagination Web Table
Unit 5	Forms
Unit 6	Footer Links
Unit 7	Tabs & Dynamic Buttons
Unit 8	Alerts & Popups
Unit 9	Mouse Hover to Scrolling Dropdown
Unit 10	Labels & Links
Unit 11	Shadow DOM

TestNG Suite Files

testng.xml

Master execution suite containing all test cases.

smoke.xml

Contains critical test cases executed after every build.

regression.xml

Contains complete regression test suite covering all functionalities.

Reports Folder

Stores:

- Extent Reports
 - Execution Logs
 - HTML Reports
-

test-output Folder

Generated automatically by TestNG.

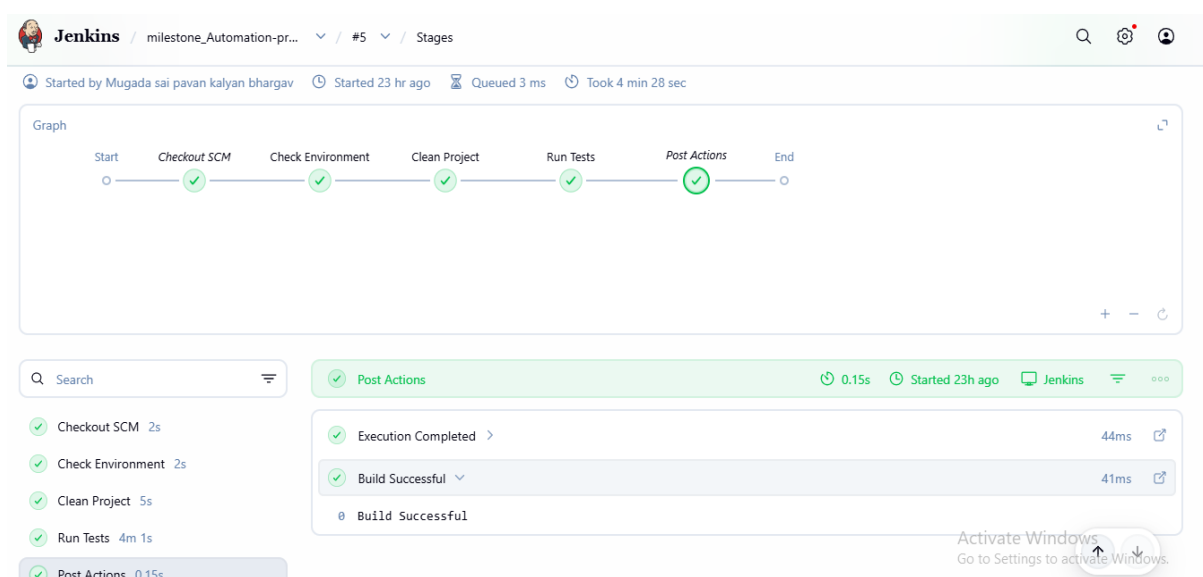
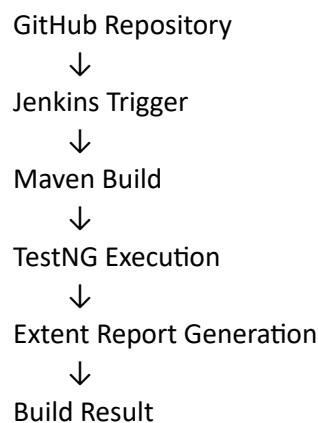
Contains:

- Emaillable Report
- TestNG Report
- Execution Summary

JENKINS INTEGRATION:

The automation framework is integrated with Jenkins to enable continuous and controlled test execution. The Jenkins pipeline is configured to automatically fetch the latest code from the GitHub repository, perform a Maven build to resolve dependencies and compile the project, and execute the TestNG test suite. The execution is initiated manually using the “Build Now” option, and the test results along with execution logs are monitored through the Jenkins console output for effective analysis and debugging.

Jenkins Pipeline Flow



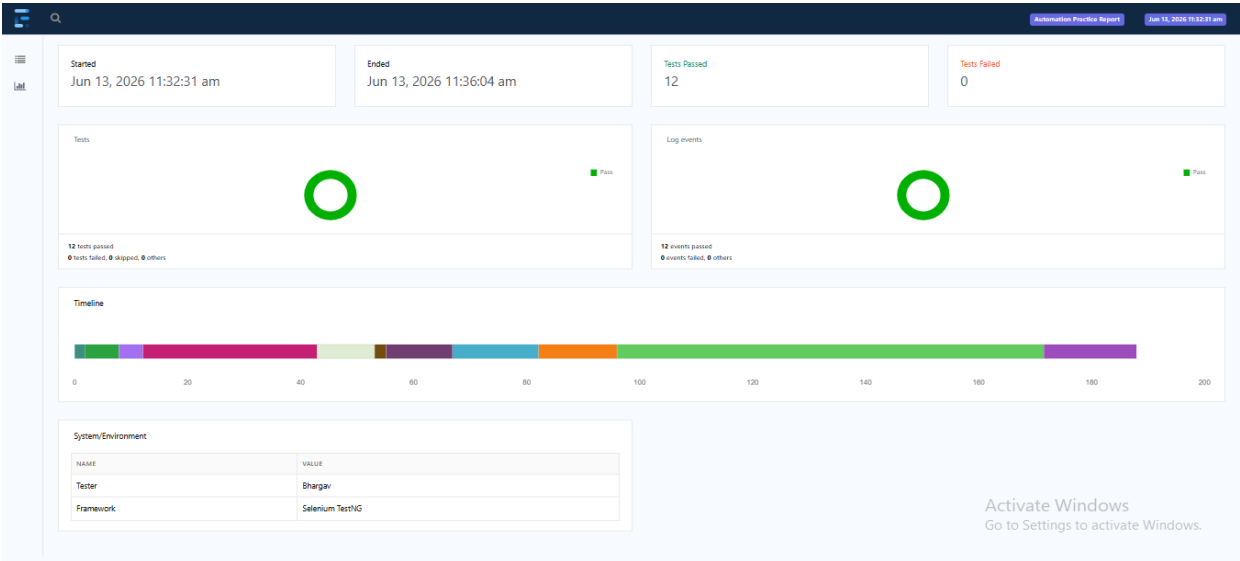
REPORTS:

Test	# Passed	# Skipped	# Retried	# Failed	Time (ms)	Included Groups	Excluded Groups
SmokeSuite							
Smoke Tests	6	0	0	0	56,022	smoke	

Class	Method	Start	Time (ms)
SmokeSuite			
SmokeTests — passed			
tests.AutomationPracticeTest	Unit1_GUIElements	1781330867138	1958
	Unit1_GUIElements		
	Unit3_FileUpload	1781330869225	4320
	Unit5_Form	1781330873667	10030
	Unit9_AdvancedMouseValidation	1781330885975	13858
	printValidLinksTest	1781330883824	2013

Unit1_GUIElements	
06.13.2026 11:37:44 am	06.13.2026 11:37:46 am 00:00:02:472 · #test-id=1
STATUS TIMESTAMP DETAILS	
Pass	11:37:46 am Test Passed

Tests	
Unit1_GUIElements	Pass
Unit1_GUIElements	Pass
Unit3_FileUpload	Pass
Unit5_Form	Pass
printValidLinksTest	Pass
Unit9_AdvancedMouseValidation	Pass



Regression Testing

Tests passed/Failed/Skipped:	5/0/0
Started on:	Sat Jun 06 08:56:09 IST 2026
Total time:	39 seconds (39273 ms)
Included groups:	smoke
Excluded groups:	

(Hover the method name to see the test class name)

PASSED TESTS			
Test method	Attribute(s)	Exception	Time (seconds)
Unit9_AdvancedMouseValidation Test class: tests.AutomationPracticeTest		14	tests.AutomationPracticeTest@7bedc48a
printValidLinksTest Test class: tests.AutomationPracticeTest		0	tests.AutomationPracticeTest@7bedc48a
Unit1_GUIElements Test class: tests.AutomationPracticeTest Parameters: Bhargav, bhargav@gmail.com, 9876543210, Kakinada		2	tests.AutomationPracticeTest@7bedc48a
Unit3_FileUpload Test class: tests.AutomationPracticeTest		0	tests.AutomationPracticeTest@7bedc48a
Unit5_Form Test class: tests.AutomationPracticeTest		6	tests.AutomationPracticeTest@7bedc48a

Test	# Passed	# Skipped	# Retried	# Failed	Time (ms)	Included Groups	Excluded Groups
RegressionSuite							
RegressionTests	6	0	0	0	150,107	regression	

Class	Method	Start	Time (ms)
RegressionSuite			
RegressionTests — passed			
tests.AutomationPracticeTest	Unit10_VerifyLaptopAndBrokenLinks	1781334303259	51593
	Unit11_ShadowDOM	1781334355024	15026
	Unit2_DatePickers	1781334235626	6392
	Unit4_PaginationWebTable	1781334242245	30776
	Unit7_TabsAndWindowsValidation	1781334273196	10369
	Unit8_AlertsAndPopupsValidation	1781334283854	19256

Unit2_DatePickers

12:33:55 pm / 00:00:06:395

Pass

Unit4_PaginationWebTable

12:34:02 pm / 00:00:30:778

Pass

Unit7_TabsAndWindowsValidation

12:34:33 pm / 00:00:10:372

Pass

Unit8_AlertsAndPopupsValidation

12:34:43 pm / 00:00:19:265

Pass

Unit10_VerifyLaptopAndBrokenLinks

12:35:03 pm / 00:00:51:604

Pass

Unit11_ShadowDOM

12:35:55 pm / 00:00:15:035

Pass

Unit2_DatePickers

06.13.2026 12:33:55 pm

06.13.2026 12:34:02 pm

00:00:06:395

#test-id=1

STATUS	TIMESTAMP	DETAILS
Pass	12:34:02 pm	Test Passed

Test	# Passed	# Skipped	# Retried	# Failed	Time (ms)	Included Groups	Excluded Groups
Default suite							
Default test	12	0	0	0	187,400		

Class	Method	Start	Time (ms)
Default suite			
Default test — passed			
tests.AutomationPracticeTest	Unit10_VerifyLaptopAndBrokenLinks	1781344663528	47172
	Unit11_ShadowDOM	1781344710782	19426
	Unit1_GUIElements	1781344555680	3451
	Unit1_GUIElements		
	Unit2_DatePickers	1781344561685	5942
	Unit3_FileUpload	1781344567772	4254
	Unit4_PaginationWebTable	1781344572172	30321
	Unit5_Form	1781344602599	9919
	Unit7_TabsAndWindowsValidation	1781344614727	13513
	Unit8_AlertsAndPopupsValidation	1781344628332	21237
	Unit9_AdvancedMouseValidation	1781344649761	13684
	printValidLinksTest	1781344612612	2028

DEFECT REPORT

A Defect Report is a document used to record and track issues identified during testing. During the automation testing of the Automation Practice Website, defects were found mainly in the Forms module, where invalid inputs such as numeric values, special characters, invalid email formats, and incorrect mobile numbers were accepted without proper validation. Each defect was documented with its description, severity, priority, steps to reproduce, expected result, and actual result.

Automation Practice Testing

Summary

List

Board

Backlog

Development

Forms

Timeline

More 2

Search ba...

MB

Filter

APT Sprint 1

Add dates

(5 work items)

000

Start sprint

APT-1

Blank Form Submission Accepted

TO DO

Jun 13

-

^

APT-2

File Upload Accepts Unsupported Fi...

TO DO

Jun 20

-

^

APT-3

Date Picker Accepts Past Invalid Date

TO DO

Jun 14

-

^

APT-4

Phone number field allows alphabeti...

TO DO

Jun 18

-

^

APT-5

Address Field Accepts Special Char...

TO DO

Jun 17

-

^

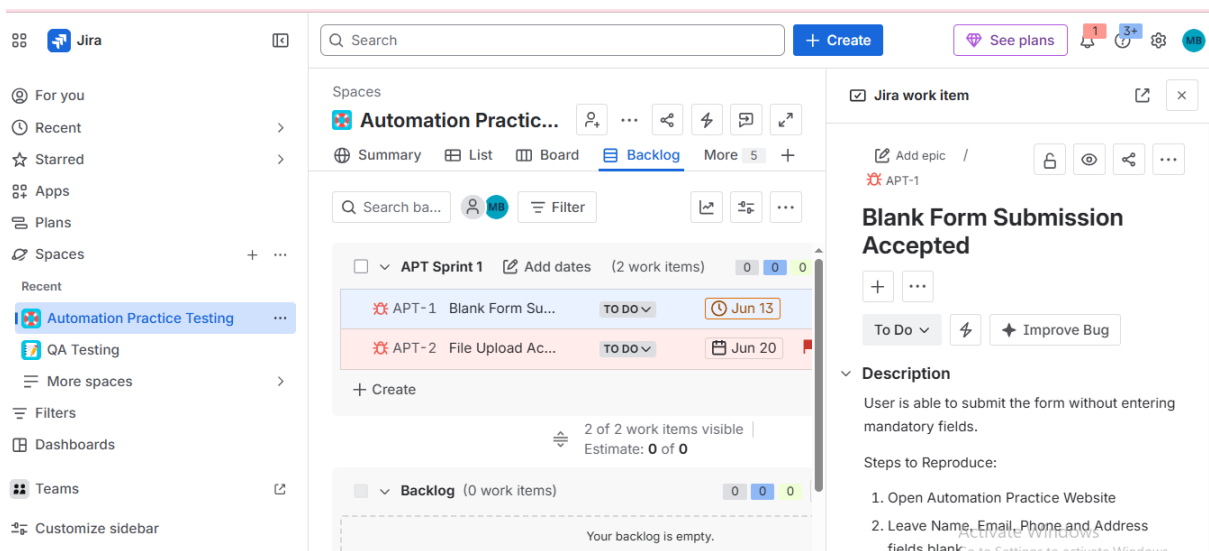
Bug_1: Blank Form Submissions Accepting without Validation

Steps:

1. Open Test Automation Practice website.
2. Leave all fields completely empty (Name, Email, Phone, Address, etc.).
3. Click the Submit button.

Expected Result: The application should block the submission, mark required fields in red, and display an appropriate "Fields cannot be blank" validation error.

Actual Result: The form submission is accepted successfully without any validation messages or data verification checks.



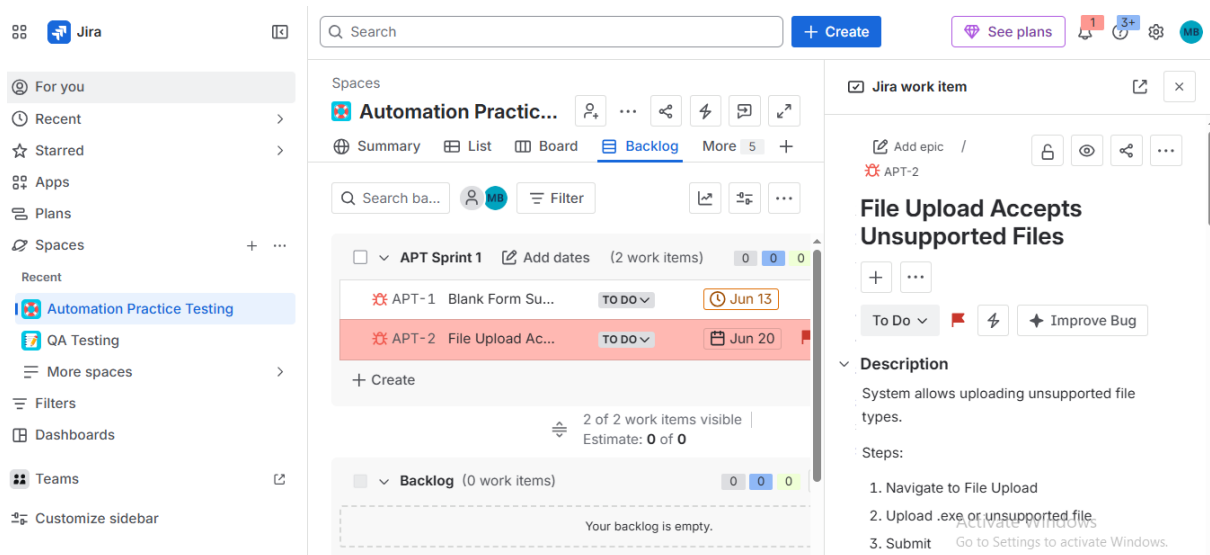
Bug_2: File Upload Accepts Unsupported File Formats

Steps:

1. Open Test Automation Practice website.
2. Locate the File Upload section.
3. Upload an invalid/unsupported file extension (e.g., an executable .exe file or system configuration file instead of standard .png/.pdf).

Expected Result: The application should validate the file format, reject the unsupported extension, and display an "Invalid file format" restriction message.

Actual Result: The file upload widget accepts and uploads unsupported file formats without any validation.



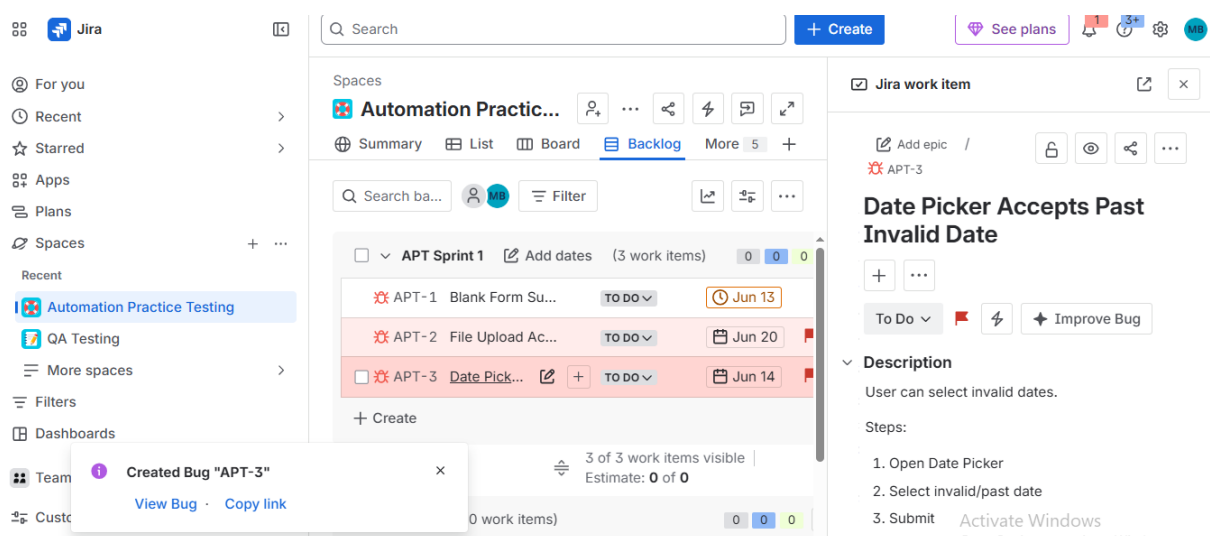
Bug_3: Date Picker Accepts Past Invalid Dates and Mismatched Ranges

Example Scenario:

- **Start Date:** 14/06/2026 (Today)
- **End Date:** 13/06/2026 (Yesterday - Past Invalid Date relative to Start Date)

Expected Result: The system logic should validate the range and reject the input. It should display a clear validation message such as *"End date cannot be earlier than start date"* or prevent selection of past dates entirely.

Actual Result: The date picker accepts the mismatched chronological range where the end date is in the past compared to the start date without throwing any validation error.



Bug_4: Phone Number field allows Alphanumeric values

Steps to Reproduce:

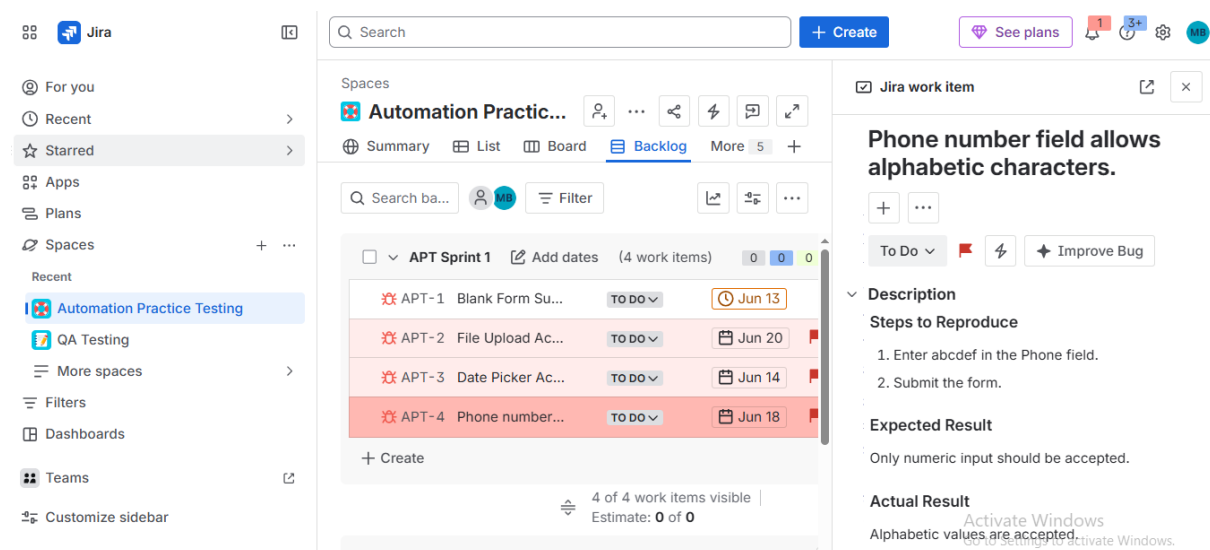
1. Open the Test Automation Practice website.
2. Scroll to the Phone / Mobile Number input field.
3. Enter an alphanumeric value (e.g., "98765ABC99") into the field.
4. Submit the form.

Example Scenario:

- **Phone Number Field Input:** 98765ABC99 (Contains alphabetic characters alongside numeric digits)

Expected Result: The field should accept only numeric data (digits 0-9). It should either prevent typing alphabetic letters completely or display a validation error message upon submission, such as *"Please enter a valid numeric phone number"*.

Actual Result: The alphanumeric input is accepted by the phone number field without any validation message, and the form submits successfully.



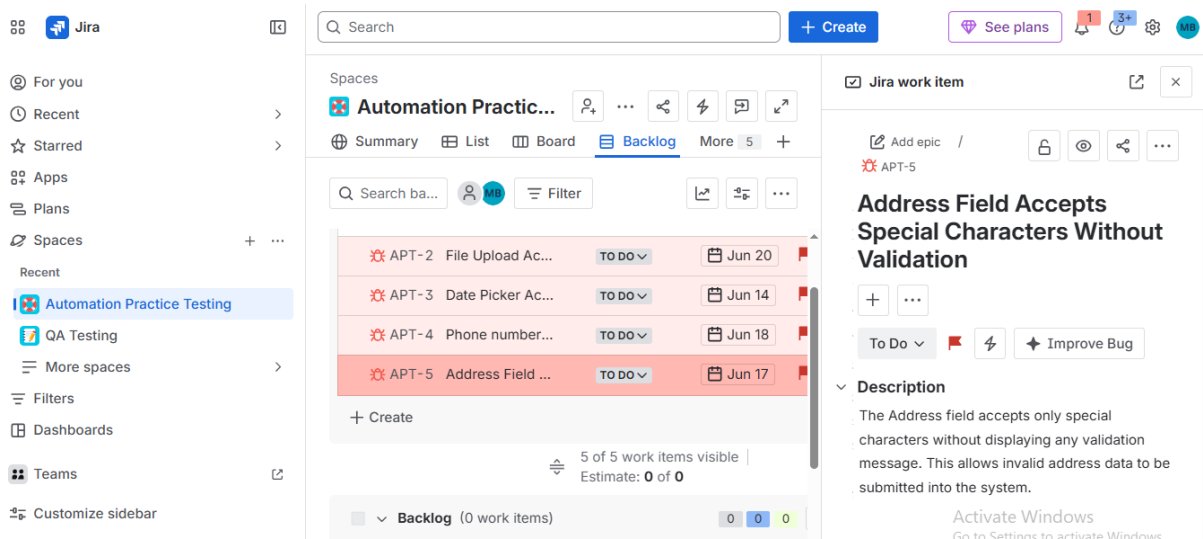
Bug_5: Address Field accepts Special Characters exclusively

Steps:

1. Open Test Automation Practice website.
2. Locate the Address text box area.
3. Enter a string consisting entirely of special characters (e.g., "!@#\$%^&*()").
4. Submit the form.

Expected Result: The address field should ensure a valid structural address format is provided, rejecting values made up entirely of non-alphanumeric special characters.

Actual Result: The special character string sequence is accepted smoothly without any form submission errors.



TESTCASE ANALYSIS REPORT

Overview

The Automation Testing Practice Framework was executed to validate the core functionalities of the Automation Testing Practice Website. Automated test cases were developed using Selenium WebDriver, Java, TestNG, Maven, Jenkins, GitHub, Extent Reports, and the Page Object Model (POM) design pattern.

The framework covers GUI Elements, Date Pickers, Upload Files, Pagination Web Tables, Forms, Footer Links, Tabs, Dynamic Buttons, Alerts & Popups, Mouse Hover Actions, Labels & Links, and Shadow DOM functionalities to ensure application quality, reliability, and efficient automated execution.

Test Execution Summary

Metric	Count
Total Test Cases	12
Executed Test Cases	12
Passed Test Cases	12

Metric	Count
Failed Test Cases	0
Blocked Test Cases	0

Module-wise Analysis

Module	Status
GUI Elements	Passed
Date Pickers	Passed
Upload Files	Passed
Pagination Web Table	Passed
Forms Validation	Passed with Defects Identified
Footer Links	Passed
Tabs Handling	Passed
Dynamic Buttons	Passed
Alerts and Popups	Passed
Mouse Hover Actions	Passed
Double Click Actions	Passed
Drag and Drop Functionality	Passed
Slider Validation	Passed
Scrolling Dropdown	Passed
Labels Validation	Passed
Links Verification	Passed
Shadow DOM Validation	Passed

Defect Analysis

Bug ID	Description	Severity
BUG-01	Name Field accepts Numeric Values without Validation	Medium
BUG-02	Name Field accepts Special Characters without Validation	Medium
BUG-03	Name Field accepts Alphanumeric Characters without Validation	Medium
BUG-04	Email Field accepts Invalid Email Format without Validation	High
BUG-05	Email Field accepts Invalid Domain Format without Validation	High
BUG-06	Mobile Number accepts less than 10 digits without Validation	High
BUG-07	Mobile Number accepts Alphanumeric Characters without Validation	High
BUG-08	Mobile Number accepts Special Characters without Validation	High
BUG-09	Mobile Number accepts more than 10 digits without Validation	High
BUG-10	Address accepts Insufficient Details without Validation	Low
BUG-11	Address accepts Numeric Only Input without Validation	Medium
BUG-12	Address accepts Special Characters Only without Validation	Medium

Test Coverage

- GUI Elements Validation
- Name Field Verification
- Radio Button Handling
- Checkbox Validation
- Sorted List Verification
- Date Picker Validation
- File Upload Functionality
- Pagination Web Table Validation
- Forms Validation
- Footer Links Verification
- Tabs and Window Handling
- Dynamic Button Validation
- Alerts and Popups Handling

- Mouse Hover Actions
 - Double Click Operations
 - Drag and Drop Operations
 - Slider Functionality
 - Scrolling Dropdown Validation
 - Labels Verification
 - Hyperlink Validation
 - Shadow DOM Handling
 - Screenshot Capture
 - Extent Reporting
 - Jenkins CI/CD Integration
-

Challenges Faced

- Jenkins Pipeline Setup
- Selenium WebDriver Version Compatibility
- Synchronization Issues
- Managing Test Dependencies
- Handling Dynamic Elements
- Shadow DOM Interaction
- Multiple Tab Handling
- Alert and Popup Handling

Conclusion

The Automation Testing Practice Framework successfully validated the major functionalities of the Automation Testing Practice Website including GUI Elements, Date Pickers, File Upload, Pagination Web Tables, Forms, Footer Links, Tabs, Dynamic Buttons, Alerts and Popups, Mouse Actions, Labels and Links, and Shadow DOM components.

The framework supports automated execution through Jenkins, utilizes TestNG for test management, generates detailed execution reports using Extent Reports, and follows the Page Object Model (POM) design pattern for better maintainability and scalability. The identified validation defects help improve the quality and reliability of the application while reducing manual testing effort through automation.

